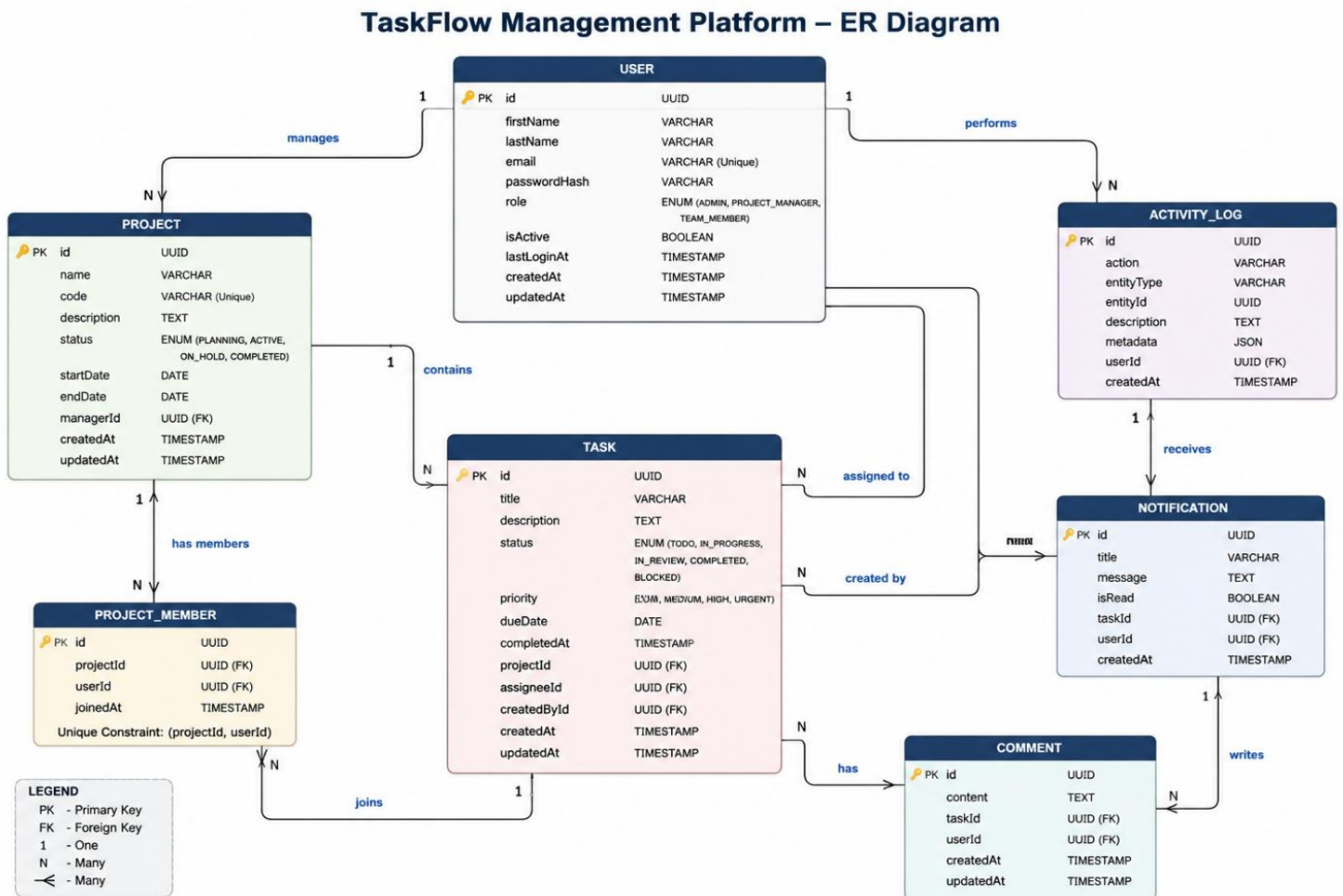


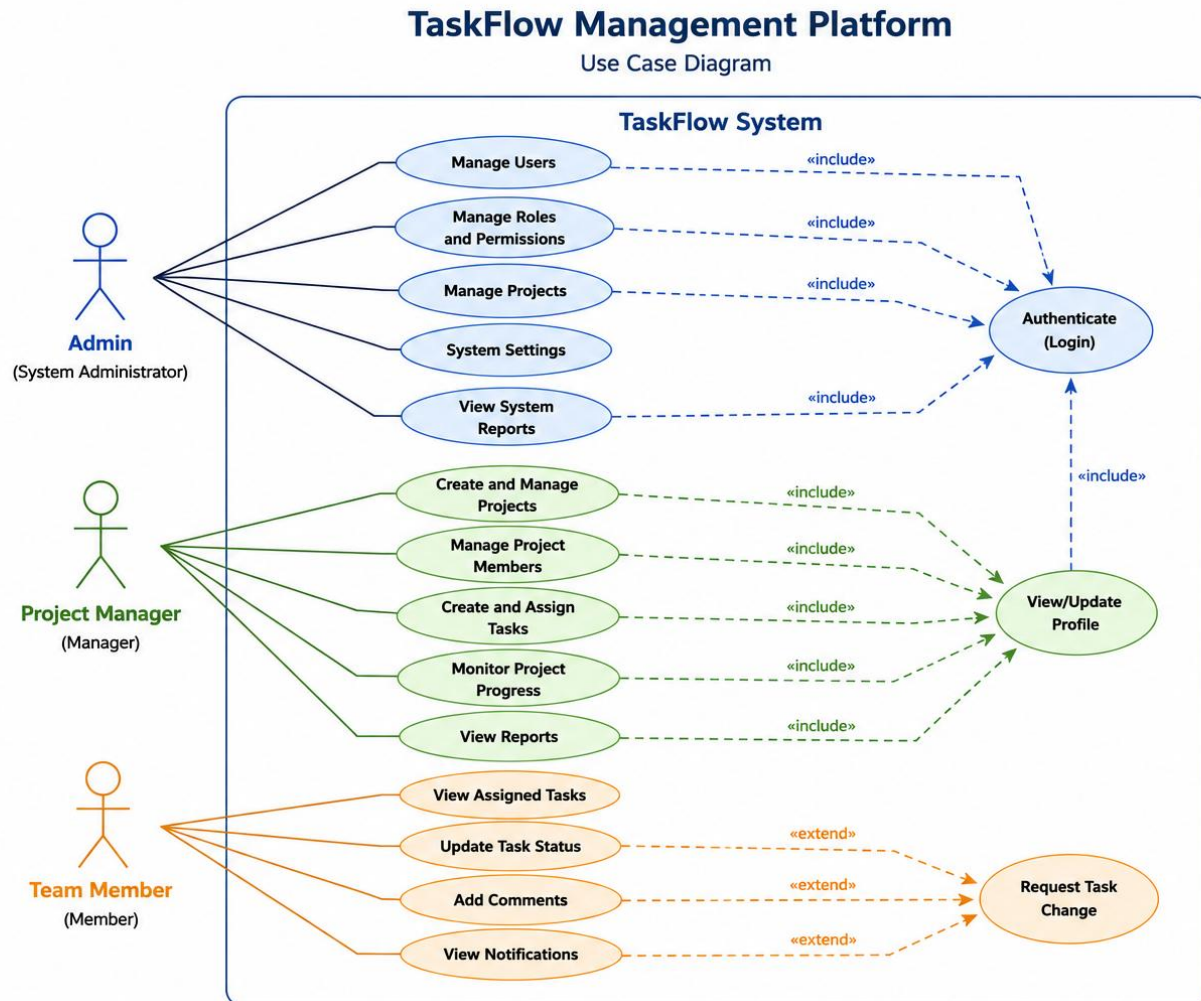
Task management platform

GitHub repository link : <https://github.com/janeeshaShehani/taskflow-management-platform.git>

Entity Relationship Diagram

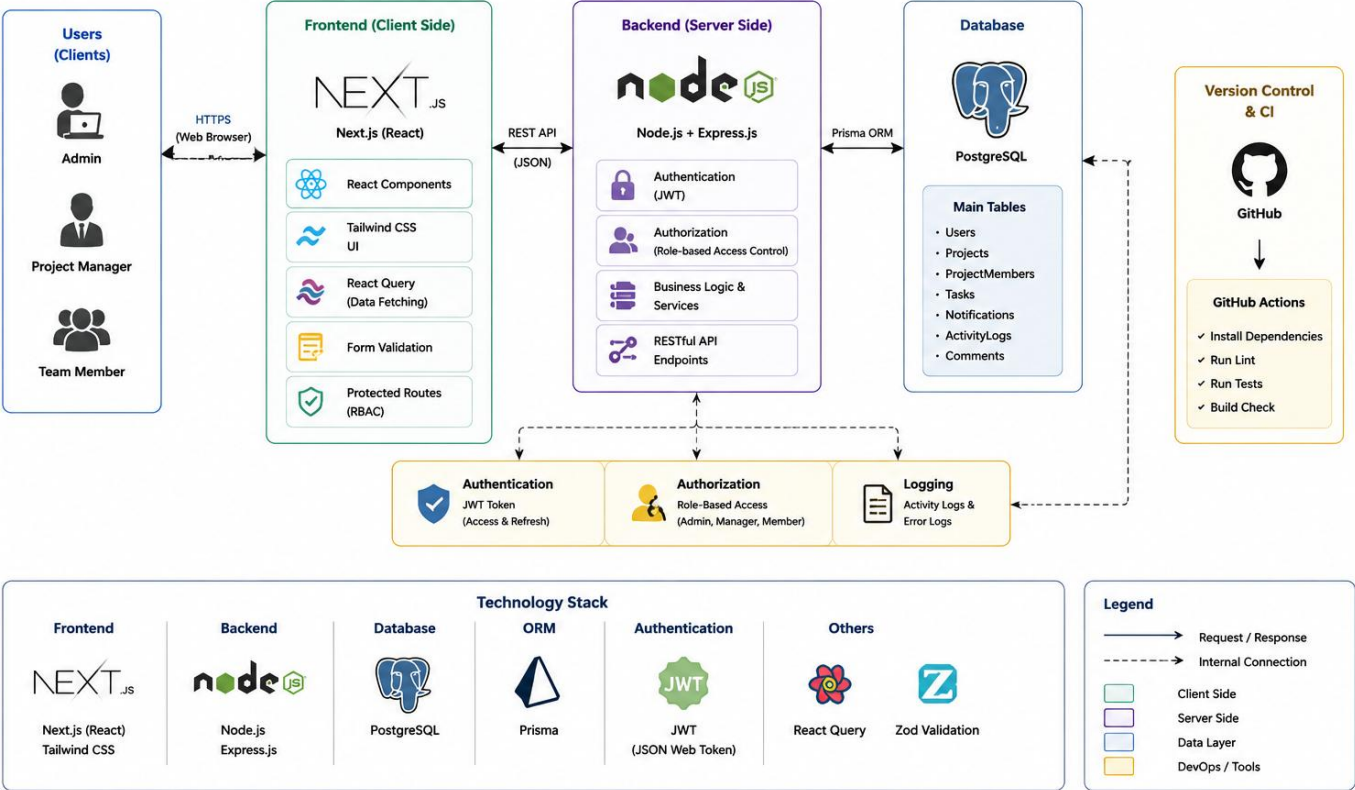


Use Case Diagram



Basic system architecture diagram

TaskFlow Management Platform
System Architecture Diagram



AI Tools Usage Statement

1. AI tools used

ChatGPT was the primary AI tool used during the development and documentation of the TaskFlow Management Platform. I used it as a support tool for explanations, debugging guidance, code review, frontend implementation suggestions, and preparation of project documentation.

No claim is made that AI independently built, tested, or submitted the project. The final implementation decisions, local setup, coding, integration, testing, Git operations, screenshots, and submission preparation were completed and verified by me.

2. Areas assisted by ChatGPT

Area	How AI assisted	My responsibility
Planning and structure	Suggested an implementation order for authentication, dashboard, users, projects, members, tasks, notifications, activity logs, responsiveness, and documentation.	I selected the final scope and implemented the features in my own repository.
Debugging	Helped explain TypeScript, React Hooks, Prisma, validation, role-permission, React Query cache, and GitHub Actions errors.	I reproduced the issues, applied the fixes, reran the application, and confirmed the results.
Frontend development	Provided example structures for Next.js pages, modals, services, types, responsive layout, role-based UI, and form validation.	I integrated the code with my existing project, adjusted it to match my backend, and tested each role.
Backend and API alignment	Helped compare frontend payloads with backend validators, routes, controllers, services, and Prisma models.	I maintained the backend codebase, database configuration, API testing, and final endpoint behavior.
Documentation	Assisted with README content, feature-completion wording, CI/CD explanation, AI-use disclosure, and diagram planning.	I reviewed the documents and will submit only information that matches the implemented project.

Version control and CI	Suggested Git commands and a GitHub Actions workflow for frontend and backend build validation.	I created the workflow, committed changes, pushed them to GitHub, and checked the workflow results.
------------------------	---	---

3. How AI output was verified

- I did not accept code only because it looked correct; I checked whether it matched the existing project structure and API contracts.
- I ran the frontend and backend locally and tested the main workflows using the browser and Postman.
- I checked role-based behavior separately for Administrator, Project Manager, and Team Member accounts.
- I reviewed TypeScript and build errors and changed AI suggestions when they did not match the actual codebase.
- I avoided placing real secrets, passwords, or database credentials in public documentation and workflow files.

4. Work completed directly by me

- Created and maintained the GitHub repository and project folder structure.
- Configured and ran the Next.js frontend, Node.js/Express backend, Prisma ORM, and PostgreSQL database.
- Executed migrations, seed/setup tasks, API tests, browser tests, commits, pushes, and build commands.
- Entered and managed project data, users, project members, and tasks.
- Reviewed the final user interface, role restrictions, responsive behavior, documentation, and submission materials.

CI/CD Workflow Explanation

Continuous Integration and Continuous Deployment (CI/CD)

The TaskFlow Management Platform uses **GitHub Actions** to implement a basic Continuous Integration (CI) workflow. The workflow automatically runs whenever changes are pushed to the **main** branch or when a pull request is created. Its primary purpose is to verify that both the frontend and backend applications build successfully before changes are merged.

Workflow Process

The CI workflow performs the following steps:

1. Repository Checkout

- GitHub Actions checks out the latest version of the project from the repository.

2. Backend Build Validation

- Sets up the required Node.js environment.
- Installs all backend dependencies using npm.
- Generates the Prisma Client.
- Builds the Node.js/Express backend to verify that there are no TypeScript or compilation errors.

3. Frontend Build Validation

- Sets up the Node.js environment.
- Installs frontend dependencies.
- Builds the Next.js application to ensure the frontend compiles successfully.

4. Build Verification

- If both frontend and backend builds complete successfully, the workflow is marked as **passed**.
- If any build step fails, GitHub Actions reports the error, allowing issues to be identified and fixed before merging new code.

Benefits

The CI workflow provides several advantages:

- Automatically validates every code change.
- Detects build and dependency errors early.
- Helps maintain code quality and project stability.
- Reduces the risk of introducing broken code into the main branch.
- Supports collaborative development by providing automated build verification.

Continuous Deployment

A Continuous Deployment (CD) pipeline has not been implemented because the application has not yet been deployed to a production environment. The current workflow focuses on Continuous Integration by automatically validating the project builds through GitHub Actions.

GitHub Actions Workflow

